

DEEPL - TP 4

Recurrent Networks

LSTM, GRU, and other memory cells

Nicolas Baskiotis - Stéphane Rivaud - Benjamin Piwowarski - Laure Soulier

2025-2026

In this practical work, we continue the study of RNNs and their application to natural language processing, by studying two problems :

1. How to process and generate variable-length sequences ?
2. How to account for longer-term dependencies (*Vanishing/exploding* gradients)

We revisit the goal of exercise 3 from TP 3 (sequence generation) as well as the dataset (Trump's pre-election speeches). You will therefore copy the code from TP 3 into the `tp4.py` file that you will gradually modify to answer the different questions.

1 Generation of variable-length sequences

Unlike the previous practical work, we consider here that each sequence is a sentence (which can end with a period, a question mark, or an exclamation point). We therefore no longer assume that sequences have a fixed size. It is therefore necessary to :

- use a special symbol that marks the end of a sequence (EOS). During training, this token must be added to each sequence to learn to predict it ;
- pad each sequence, i.e., add a null character (BLANK) as many times as necessary so that the sequences in the same batch all have the same length ; this character must be ignored during training (see question 2).

Question 1

In `textloader.py`, the `TextDataset` class is such that :

- each example is a sentence ("." has been used as delimiter) ;
- the dataset size is the number of sentences in the corpus ;
- the returned sentence is in the form of a sequence of integers, each integer encoding a character (application of the `string2code` function).

This class returns examples of variable length. It is therefore necessary to specify to the DataLoader how to group the examples to construct a batch. This is the role of the `collate_fn` argument of the DataLoader constructor, which takes a function as parameter.

Define a `pad_collate_fn` function that prepares a batch tensor (of size `length x batch size` where `length` is the maximum sequence length in the batch) from a list of examples from `TextDataset` : it must add the EOS symbol code at the end of each example and pad the sequences with the null character code.

Run `textloader.py` to verify that everything works properly.

Moreover, care must be taken not to include padding when calculating the cost - here the maximum likelihood : for this, the most common solution¹ is to use a binary mask (0 when the character is null, 1 otherwise) which is multiplied with the log-probabilities before adding them (the `reduce="none"` parameter in a cost function - particularly for cross entropy - allows obtaining the cost for each element rather than the average).

Question 2

Create your cost function `maskedCrossEntropy(output, target, padcar)` in `tp4.py` that allows calculating the cost without taking null characters into account based on the obtained output `output`, the desired output `target`, and the padding character code `padcar`. You will be careful not to use any loops for this calculation.

The last modification is at the generation level.

Question 3

In `generate.py`, implement a generation function so as to generate sequences until encountering the EOS character (remember to provide a maximum size nonetheless!). You can plan to do random generation in the obtained distribution or deterministic generation by choosing the most probable character at each time step.

During TP3, you used one-hot encoding for each character, followed by a linear module. Torch's `nn.Embedding` module allows combining these two steps to avoid the (unnecessary and costly) creation of one-hot vectors.

1. Until the appearance of `packedsequence` which you will not use in this practical work.

Question 4

Modify your code to use *embeddings* directly through the `nn.Embeddings` module, without manually going through a *one-hot* representation.

2 Accounting for long-range dependencies : LSTM and GRU

Recurrent neural networks (RNN) are a type of architecture favored for processing sequences by “encoding” a sequence iteratively (see TP 4), through a function f that computes the next state through an encoding step : $\mathbf{h}_t = f(\mathbf{x}_t, \mathbf{h}_{t-1})$.

RNNs struggle to capture long-term dependencies. This is due to the fact that the gradient is very unstable when going back a few time steps ; simplifying to the extreme : the contribution of input t for step $t + k$ will then be of the order of (see [3])

$$\|\mathbf{W}^k\|^2 = \sum |\lambda_i|^{2k}$$

In this practical work, we will study variants of RNNs, and in particular Long-Short Term Memories/LSTMs [2] and Gated Recurrent Units/GRU [1] (see also this post).

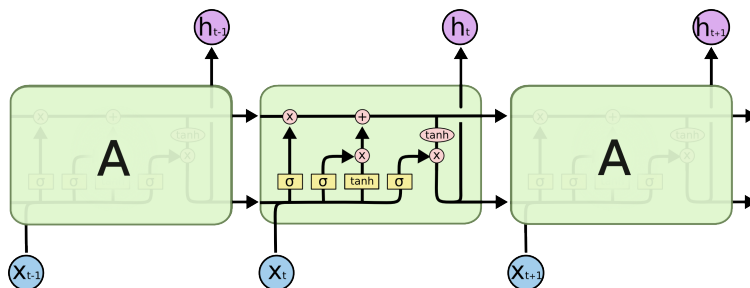


FIGURE 1 – An LSTM (Colah’s blog)

LSTMs are defined by an external state h_t (analogous to those of usual RNNs) and an internal state C_t that represents the "long-term" memory of the network. At each time step, C_t is updated based on the previous internal state h_{t-1} and the input x_t by specifying what should be "forgotten" from the past and what should be "retained" for the future. The new external state is calculated from the new internal state. This writing mechanism relies on the notion of *gate* borrowed from logic, which allows masking part of the signal that has little interest. In the case of neural networks, a gate is a continuous function (and not discrete as in logic), often a linear layer followed by a sigmoid activation (which therefore produces an output between 0 and 1).

The evolution of internal and external states is defined by the following equations (Figure 1) which relies on three gates (forget, input, and output) :

f_t	$= \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	Gate (forget)
i_t	$= \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	Gate (input)
C_t	$= f_t \otimes C_{t-1} + i_t \otimes \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$	Update (internal mem.)
o_t	$= \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	Gate (output)
h_t	$= o_t \otimes \tanh(C_t)$	Output

where $[a, b]$ is a vector concatenation and $\otimes$ is an element-wise product (Hadamard).

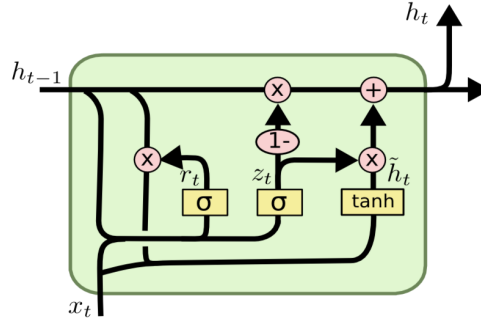


FIGURE 2 – A GRU (Colah’s blog)

GRUs are simplifications of LSTMs (there are others) – the internal and external state is the same, the input/forget gates are merged (with $i_t := 1 - f_t$); the equations that define a GRU (Figure 2) are :

$$\begin{aligned}
 z_t &= \sigma(W_z \cdot [h_{t-1}, x_t]) \\
 r_t &= \sigma(W_r \cdot [h_{t-1}, x_t]) \\
 h_t &= (1 - z_t) \otimes h_{t-1} + z_t \otimes \tanh(W \cdot [r_t \otimes h_{t-1}, x_t])
 \end{aligned}$$

Question 5

Implement a GRU then an LSTM. Note that for LSTMs, there is an internal state. Compare the results in likelihood (loss) as well as qualitatively (generated sequences).

Use tensorboard to monitor the magnitude of gradients as well as the evolution of the values of the different gates. Use `add_histogram` for this. If you have stability problems and the magnitudes of the gradients are large, use “gradient clipping”.

3 Beam-search

During your generation attempts, you will observe that by taking the argmax at each time step you will very rarely obtain intelligible sentences (you are in fact making a greedy

approximation of the maximum likelihood). Initially, you can sample at each time step from the inferred distribution, but the result will not be much better. The usual solution consists of using a “beam search” to approximate the argmax over the entire generated sequence : beam search consists of maintaining at all times t a set of K sequences (and their associated log-probability) ; at step $t + 1$, we generate for each of the sequences s the K most probable symbols given s . Then we select the K most probable sequences of size $t + 1$ (and re-iterate).

Question 6

Implement a beam-search for sentence generation, and qualitatively compare the obtained results.

The quality of generated sequences can be further improved by using sampling techniques such as Nucleus Sampling [4], which consists of defining the generation probability by considering only the most probable characters, by defining a threshold α (a hyperparameter, e.g., 0.95) that allows selecting only the outputs that best cover this probability mass. Formally, if $I_\alpha(p, s)$ is the smallest set of symbols such that

$$\sum_{I_\alpha(p,s)} p(y|s) \geq \alpha$$

, the *nucleus* probability is defined as

$$p_{\text{nucleus } K}(y|s_t) = \mathbb{1}[y \in I_\alpha(p, s_t)] \times \frac{p(y|s_t)}{\sum_{y' \in I_\alpha(p, s_t)} p(y'|s_t)}$$

Question 7

Implement beam search using Nucleus sampling.

Références

- [1] Kyunghyun CHO et al. “Learning Phrase Representations Using RNN Encoder-Decoder for Statistical Machine Translation”. In : 2014.
- [2] Sepp HOCHREITER et Jürgen SCHMIDHUBER. “Long Short-Term Memory”. In : 2006.
- [3] Sepp HOCHREITER et al. “Gradient Flow in Recurrent Nets : The Difficulty of Learning Long-Term Dependencies”. In : (2001).
- [4] Ari HOLTZMAN et al. “The Curious Case of Neural Text Degeneration”. In : International Conference on Learning Representations. 25 sept. 2019.